

Raspberry Pi OS

Download, Verify and Flash — the exact procedure for a Raspberry Pi 4 (4 GB)

docs/01_Raspberry_Pi_OS_Download_and_Flash.pdf | Course repository, document 1 of 2 | Seminar: Real-Time Linux — Fundamentals and Practical Applications | Trainer: Gaëtan Pandji | v1.1 · 20 August 2026 · facts checked against raspberrypi.com on that date

Where you are. This is document 1 of 2 in the course repository. Everything referenced here — scripts, exercises, acceptance criteria — lives in the same repo, and you have read access to all of it.

```
rt-linux-seminar/
├── README.md          <- start here
├── docs/
│   ├── 01_Raspberry_Pi_OS_Download_and_Flash.pdf  <- THIS DOCUMENT
│   └── 02_PREEMPT_RT_Setup_on_Raspberry_Pi_4.pdf
├── scripts/
│   ├── provision-pi.sh      baseline setup (section 6 of this document)
│   ├── verify.sh           the checklist in section 7, automated
│   ├── build-rt-kernel.sh  document 2
│   ├── switch-kernel.sh   document 2
│   └── latency_showdown.sh day 1, live demo
├── exercises/
│   ├── AUFGABE_Stufe1.md, stage1_naive.c, stage1_reference.c
│   └── DEFINITION_OF_DONE.md      acceptance criteria per project stage
```

What this document is. A complete, copy-paste procedure to go from a blank microSD card to a booted, updated, SSH-reachable Raspberry Pi 4 with I²C enabled. It stops exactly where Document 2 begins.

Result you can verify: `uname -a` reports a 6.18 kernel, `i2cdetect -y 1` lists your sensor, and you are logged in over SSH.

Time: 25-40 minutes, most of it download and first-boot expansion.

When to do it: *before* day 1, not during it. If anything in section 7 fails, message the trainer with the output — we fix it in the technical check session, not in the seminar.

Everything in this document is reversible. Nothing here touches your work computer beyond installing Raspberry Pi Imager. All destructive operations happen on the microSD card, which you can re-flash at any time in ten minutes. If you break the Pi, you have lost nothing but the card contents — that is deliberate, and section 8 tells you how to snapshot a working state so re-flashing costs you nothing at all.

1. Before you start — what you need

Item	Requirement and why
Raspberry Pi 4 Model B, 4 GB	BCM2711, Cortex-A72. The RT kernel history on this SoC is longer and better tested than on the Pi 5 (RP1 I/O).
Official USB-C power supply, 5 V / 3 A	Under-voltage causes clock throttling, which shows up later as latency outliers. Do not use a phone charger.
microSD card, 32-64 GB, class A2	A kernel build tree plus modules needs real space. A1/cheap cards make native kernel builds painfully slow.
Active cooler or fan case	Thermal throttling is itself a determinism killer — and a teaching example in the course.
Ethernet cable (Cat 5e/6)	Wi-Fi adds jitter and DHCP surprises. Use wired for anything you will measure.
Card reader on your PC	Also your recovery path if a custom kernel ever fails to boot (see Document 2, §11).
I ² C sensor (e.g. BME280, Qwiic/STEMMA QT)	Needed for the course project. Pluggable connector = nobody mis-wires 3.3 V into a data pin over a video call.

2. Choose the right image

The download page lists four families. For this purpose the choice is not a matter of taste:

Image	Verdict	Reason
Raspberry Pi OS Lite (64-bit)	Use this	~501 MB download, ~2.8 GB on card. No desktop = no compositor, no browser, no background GUI services competing for CPU. 64-bit is required: <code>CONFIG_PREEMPT_RT</code> is only offered on architectures that select <code>ARCH_SUPPORTS_RT</code> , and on Raspberry Pi that means arm64.
Raspberry Pi OS (64-bit, with desktop)	Avoid	Adds a Wayland desktop you will never see over SSH, and adds noise to every latency measurement.
Raspberry Pi OS (32-bit)	Do not use	armhf userland. Wrong architecture for the RT work in Document 2.
Raspberry Pi OS Legacy (Bookworm, 6.12)	Fallback only	Debian 12 with kernel 6.12 — also RT-capable (<code>PREEMPT_RT</code> was mainlined in 6.12). Use only if something in Trixie blocks you.

Current release at time of writing (19 Aug 2026): Raspberry Pi OS, build dated **2026-06-18**, Debian 13 “Trixie”, **Linux 6.18.34 LTS** (up from 6.12.75 in the previous builds). Both Trixie and the Legacy Bookworm line were refreshed on the same day.

3. Route A — Raspberry Pi Imager (recommended)

Imager downloads, verifies and writes in one pass, and — the part that actually matters for a remote course — pre-seeds the user account, SSH key and network *before* first boot. No keyboard or monitor is ever needed.

Steps

1. **1** Download Raspberry Pi Imager from <https://www.raspberrypi.com/software/> and install it (Windows, macOS, Ubuntu/Debian).
2. **2** Insert the microSD card into your PC.
3. **3** Open Imager. **Choose device** → *Raspberry Pi 4*.
4. **4** **Choose OS** → *Raspberry Pi OS (other)* → **Raspberry Pi OS Lite (64-bit)**.
5. **5** **Choose storage** → select the card. Read the size twice. Imager will erase it without asking again.
6. **6** Click **Next**. When asked “Would you like to apply OS customisation settings?” choose **Edit Settings**.
7. **7** On the **General** tab set:
 - Hostname: `rtpi` (or `rtpi-trainer` / `rtpi-student` to keep two boards apart)
 - Username and password — do **not** use `pi`; pick something like `rt`
 - Wireless LAN: only if you truly have no Ethernet. Set the correct country code.
 - Locale: time zone `Europe/Berlin`, keyboard layout `de`
8. **8** On the **Services** tab: enable **SSH**, and choose **Allow public-key authentication only**. Paste your public key (`~/.ssh/id_ed25519.pub`). If you have no key yet: `ssh-keygen -t ed25519`.
9. **9** On the **Options** tab: enable *Eject media when finished*. Save, then **Yes** to apply, then **Yes** to erase.
10. **10** Wait for “Write Successful”. Imager verifies the written data automatically — do not cancel that phase.

Why public-key-only matters here. If this board is going to a participant's office network, password SSH on a device you cannot see is a liability. Keys cost you thirty seconds and remove the whole class of problem.

4. Route B — manual download and SHA-256 verification

Use this when you need a reproducible, archived copy of one exact image — which you do, if the trainer and the participant must run identical systems so that live latency figures match.

4.1 Download

Go to <https://www.raspberrypi.com/software/operating-systems/>, section **Raspberry Pi OS (64-bit) → Raspberry Pi OS Lite**, and click **Download**. The direct URL of the current build is:

```
https://downloads.raspberrypi.com/raspbian_lite_arm64/images/\
raspios_lite_arm64-2026-06-19/2026-06-18-raspbian-trixie-arm64-lite.img.xz
```

Or from the command line: `wget <url>`. The page also offers a torrent and an SBOM. Older builds stay available under *View archive* — that is how you pin an exact image months later.

4.2 Verify the checksum — do not skip this

The page prints a SHA-256 hash next to each image. Compare it with the hash of your downloaded file.

```
# Linux / Raspberry Pi OS
sha256sum 2026-06-18-raspbian-trixie-arm64-lite.img.xz

# macOS
shasum -a 256 2026-06-18-raspbian-trixie-arm64-lite.img.xz

# Windows PowerShell
Get-FileHash .\2026-06-18-raspbian-trixie-arm64-lite.img.xz -Algorithm SHA256
```

Expected value for the 2026-06-18 Lite (64-bit) image:

```
acff736ca7945e3b305f07cda4abdb870910e12634991da69783611756e381b3
```

If the hash differs, stop. Delete the file and download again. A truncated image is the single most common cause of “my Pi boots to a rainbow screen and hangs”. Two minutes of checking here saves an hour of confused debugging later.

Reference hashes for the same 2026-06-18 build, in case you deliberately choose a different variant:

Variant (64-bit)	SHA-256
Lite (recommended)	acff736ca7945e3b305f07cda4abdb870910e12634991da69783611756e381b3
Desktop	123287c05f27b0eebd8f65456f6369b8f6635fa50a3d440a4f9f6223bf58c8e2
Full	2ca7ec6ac0e43595774ebf1061334a00f9dc0d1ea879b895a174a52ea8e32ec3
Legacy Lite (Bookworm, 6.12)	df4c6fb0b625204a09ce587f35f01b4d6375cb3ee0f05df71a0eca6c7aeb8f8f

4.3 Write the image

Preferred: open Imager, **Choose OS → Use custom**, select your `.img.xz` file, then continue exactly as in Route A from step 5 — you keep the OS-customisation screen, which is the whole reason to use Imager at all.

Command line alternative (Linux, expert):

```
lsblk          # identify the card, e.g. /dev/sdb - CHECK THE SIZE
sudo umount /dev/sdb* # unmount any auto-mounted partitions
xzcat 2026-06-18-raspbian-trixie-arm64-lite.img.xz | sudo dd of=/dev/sdb bs=4M \
    status=progress conv=fsync
sync
```

dd will silently destroy the wrong disk. Confirm the device node with `lsblk` immediately before and after inserting the card. Write to the *disk* (`/dev/sdb`), never to a partition (`/dev/sdb1`). With this route you must enable SSH manually: see §5.1.

4.4 Manual headless enablement (only needed after the dd route)

Re-insert the card; the FAT partition mounts as `bootfs`.

```
# 1) Enable SSH
touch /media/$USER/bootfs/ssh

# 2) Create the first user (password hash, not plaintext)
echo "rt:$(echo 'YourPassword' | openssl passwd -6 -stdin)" \
    > /media/$USER/bootfs/userconf.txt
```

5. First boot

1. Insert the card, connect Ethernet, then connect power. The green LED should flicker within a few seconds.
2. The first boot resizes the root filesystem and reboots once. Allow 1–2 minutes before expecting SSH.
3. Connect:

```
ssh rt@rtpi.local      # mDNS - works on most home/office LANs
ssh rt@192.168.1.42    # fallback: find the IP in your router's DHCP list
```

If `rtpi.local` does not resolve, the network is blocking mDNS (guest Wi-Fi, client isolation, corporate VLAN). Fall back to the IP address from your router. If neither works, that is what the pre-installed Tailscale path is for — see `README.md`, section “Network fallback”. Please establish reachability **before** day 1 and tell the trainer if you cannot.

6. Baseline configuration

```
# Update the whole system (full-upgrade handles dependency changes; upgrade does not)
sudo apt update
sudo apt full-upgrade -y

# Enable I2C non-interactively
sudo raspi-config nonint do_i2c 0

# Optional: SPI as well, if the course project uses it
sudo raspi-config nonint do_spi 0

# Tools you will need on Day 1 and Day 2
sudo apt install -y i2c-tools git vim tmux htop stress-ng \
    bc bison flex libssl-dev make libncurses-dev

sudo reboot
```

Alternatively, run `sudo raspi-config` → *Interface Options* → *I2C* → *Yes*. All of the above is also available as one command from the repository:

```
git clone <repository-url> ~/rt-linux-seminar
sudo ~/rt-linux-seminar/scripts/provision-pi.sh
```

7. Verification checklist

Run all of them. If any line disagrees, fix it before touching Document 2 — debugging a custom kernel on top of a broken base wastes an evening. The repository automates this:

```
~/rt-linux-seminar/scripts/verify.sh
```

If it reports a failure you cannot resolve, paste the full output into the course channel. That output is more useful to the trainer than a description of the symptom.

Command	Expected
<code>uname -a</code>	Contains 6.18, aarch64, SMP PREEMPT (not yet PREEMPT_RT — that is Document 2)
<code>cat /etc/os-release</code>	VERSION_CODENAME=trixie, Debian 13
<code>cat /proc/device-tree/model</code>	Raspberry Pi 4 Model B Rev 1.x
<code>nproc</code> and <code>free -h</code>	4 and roughly 3.7Gi total
<code>ls /dev/i2c-*</code>	/dev/i2c-1 present
<code>i2cdetect -y 1</code>	Sensor address visible, e.g. 76 or 77 for a BME280
<code>vcgencmd get_throttled</code>	throttled=0x0 — anything else means power or heat problems
<code>df -h /</code>	Full card capacity, i.e. the filesystem expanded

8. Make it reproducible — back up the golden image

Once the board verifies clean, snapshot it. Rebuilding this state by hand under time pressure is not a plan.

```
# On a Linux PC, card in the reader (/dev/sdb):
sudo dd if=/dev/sdb bs=4M status=progress | xz -T0 -3 > rtpi-golden-$(date +%F).img.xz

# Restore later:
xzcat rtpi-golden-2026-08-19.img.xz | sudo dd of=/dev/sdb bs=4M status=progress conv=fsync
```

The image is card-size-dependent. Shrink it with `pishrink` if you need it to fit smaller cards. Store the SHA-256 of your own image alongside it, the same way Raspberry Pi does.

9. Troubleshooting

Symptom	Cause and fix
Green LED flashes 4 long / 4 short	Firmware cannot find <code>start4.elf</code> — bad or incomplete write. Re-flash, verify the SHA-256 first.
Rainbow screen, then nothing	Corrupt image or a failing card. Re-download, re-flash, and try a different card before blaming the Pi.
SSH: Connection refused	SSH not enabled. Re-flash with Imager customisation, or create the empty <code>ssh</code> file on the boot partition.
SSH: Permission denied (publickey)	Wrong key pasted into Imager. Mount the card and inspect <code>/home/<user>/.ssh/authorized_keys</code> on the root partition.
<code>rtpi.local</code> unresolvable	mDNS blocked. Use the IP from the router, or the Tailscale fallback.
<code>i2cdetect</code> shows nothing	I ² C not enabled (<code>ls /dev/i2c-*</code>), sensor unpowered, or wrong bus. On the 40-pin header, bus 1 is GPIO2 (SDA) / GPIO3 (SCL).
<code>throttled=0x50000</code> or similar	Under-voltage or over-temperature has occurred. Replace the PSU, add cooling. Do not measure latency on a throttling board.

10. Scope, assumptions and known limits

Stated plainly so you know what this document does and does not cover.

- **Verified fact:** URLs, SHA-256 hashes, image variants, kernel version and package names were checked against raspberrypi.com on 19 August 2026. **Raspberry Pi re-releases images without changing the marketing version number** — always read the hash from the live page rather than trusting this PDF. If the hash on the page differs from section 4.2, the page is right and this document is stale; please open an issue in the repository.
- **Assumption:** you boot from microSD, not USB SSD or NVMe. USB boot works, but it changes the recovery procedure in Document 2, section 11. Tell the trainer if you intend to boot from USB.
- **Assumption:** your network permits SSH from your computer to a device on the same LAN. Corporate networks with client isolation break this. Establish it before day 1.
- **Assumption:** an I²C sensor on the 40-pin header, addressed on bus 1. A different sensor or bus changes section 7 but nothing else.
- **Design decision, with reasoning:** Lite rather than Desktop. Every GUI service is a background CPU consumer and therefore a source of latency outliers in the measurements we make on day 2 and day 3. This is not a preference — a desktop image will visibly degrade your `cyclicttest` results.
- **Not covered here:** the real-time kernel itself. That is Document 2. Do not start it until section 7 passes.

11. Contributing and change log

If a command in this document fails on your machine, that is a defect in the document, not in you. Open an issue in the repository with: the command, the full output, your OS and the image build date. Corrections are welcome as pull requests against `docs/`.

Version	Date	Change
v1.1	20 Aug 2026	Pagination fix: lists and their headings no longer split across pages.
v1.0	19 Aug 2026	Initial release. Based on Raspberry Pi OS build 2026-06-18 (Debian 13 Trixie, Linux 6.18.34).

Next: `docs/02_PREEMPT_RT_Setup_on_Raspberry_Pi_4.pdf` — building and installing the real-time kernel.

Repository index: `README.md` · Acceptance criteria: `DEFINITION_OF_DONE.md` · Questions: course channel or repository issues.